



Real-time Demo Workshop

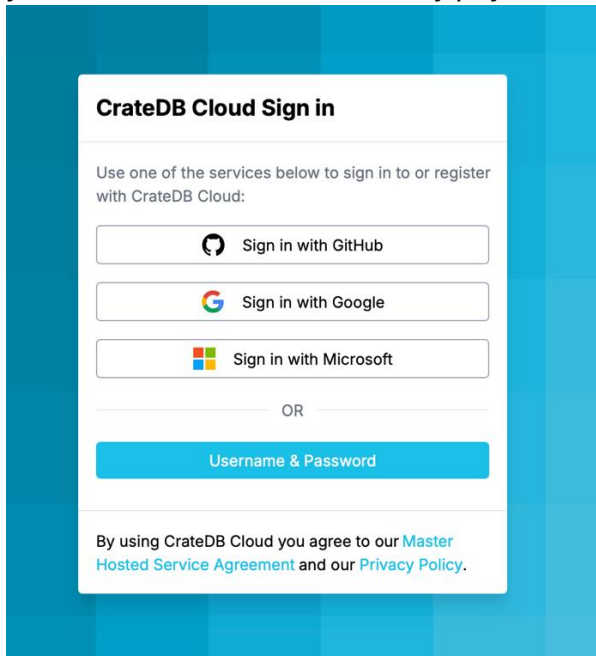
Table of Contents

Table of Contents	1
Workshop Setup Instructions	2
Creating a CrateDB Cluster	3
View Grafana dashboards	4
Exercises - Query	7
What is the time range?	8
Distribution of temperature	8
Average temperature per day	8
10 hottest points	9
Are we missing any data points?	9
Wind speed calculations	10
What are the top 10 windiest observations?	11
Wind direction calculated using trigonometry	12
Exercises - Geo	12
How to extract latitude & longitude from a GEO_POINT	13
Distance calculations	14
Exercises – Correlation	14
Is pressure and temperate related?	15
Is pressure and wind speed related?	15
Exercise – JSON object model	15

Workshop Setup Instructions

Creating a CrateDB Cluster

1. Go to <https://cratedb.com>
2. Click on the Start Free button
3. You have several options to create an account, use the one that is appropriate for you. You won't need to enter any payment details.



4. Once you have an account, log in and deploy a CRFREE cluster

Configure your new cluster

Shared
Single-node, suitable for non-production use cases with up to 8 shared vCPUs

Dedicated
Scalable up to 9 nodes, ideal for production workloads with up to 144 vCPUs

Custom
Customizable for large-scale needs, offers any cluster size and custom compute options

Shared tier clusters operate on a single node without replication, sharing vCPUs with other clusters. Performance may vary depending on the overall load, and usage is based on a fair-use principle.
[Learn more about cluster types](#)

Cloud Provider

aws

A

G

Region

aws

AWS US-East (N.Virginia)
eks1.us-east-1.aws

[Request a new region](#)

Node compute size

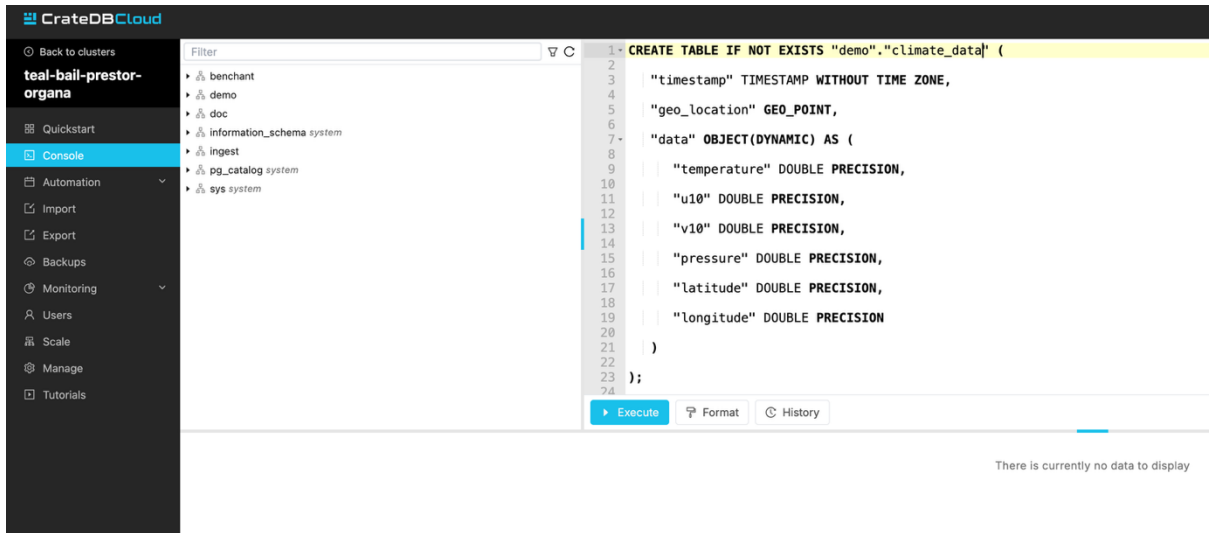
CRFREE	Up to 2 vCPU	2 GiB RAM	FREE
S2	Up to 2 vCPU	2 GiB RAM	\$0.079 per hour
S4	Up to 3 vCPU	4 GiB RAM	\$0.158 per hour
S6	Up to 4 vCPU	6 GiB RAM	\$0.238 per hour
S12	Up to 8 vCPU	12 GiB RAM	\$0.475 per hour

Node storage size

8 GiB **FREE**

- Select a Shared cluster (this is the only way to get a free cluster)
- Select any cloud provider
- Select any region

- Ensure CRFREE is selected as compute size
 - Click Deploy Cluster to create it, which will take a short amount of time
5. It's **very** important that you save the login credentials, so download or make a note now before moving on.
 6. We need to create the destination table for the climate data, run the following in the Console



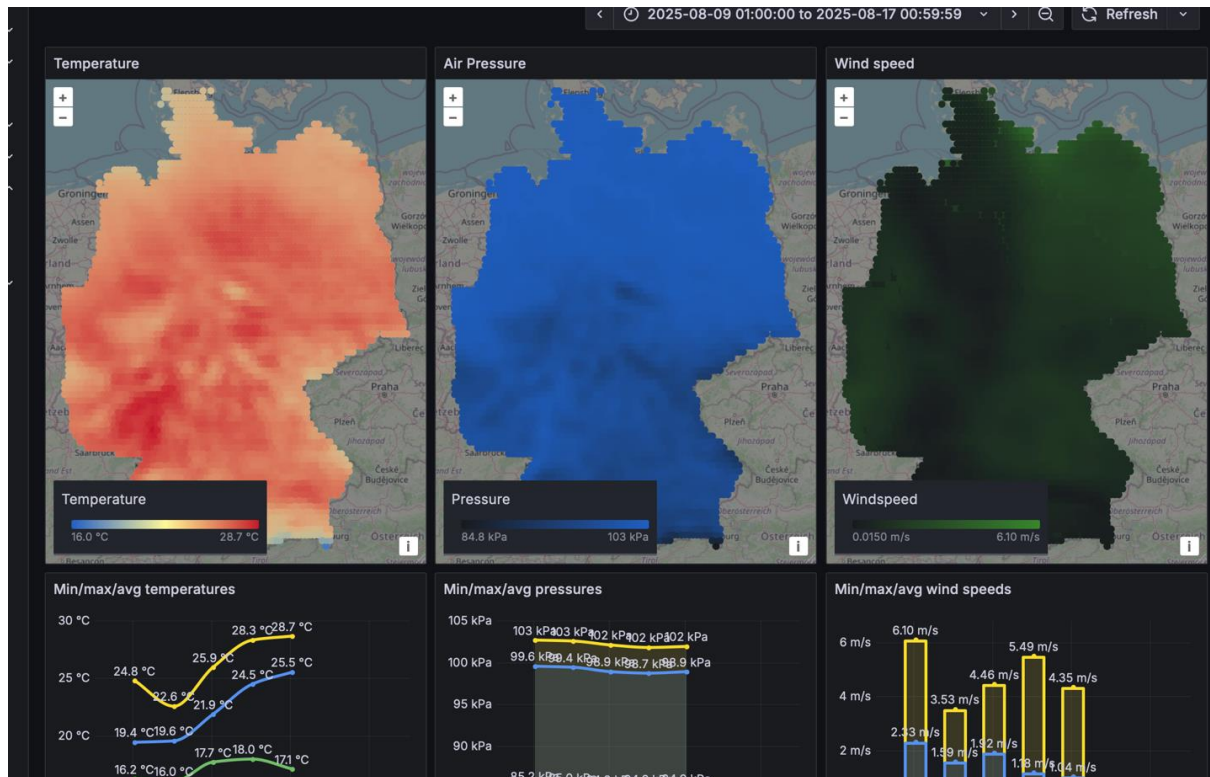
7. Got to "Console" and paste the create table statement in and click "Execute"

```
CREATE TABLE IF NOT EXISTS "demo"."climate_data" (
  "timestamp" TIMESTAMP WITHOUT TIME ZONE,
  "geo_location" GEO_POINT,
  "data" OBJECT(DYNAMIC) AS (
    "temperature" DOUBLE PRECISION,
    "u10" DOUBLE PRECISION,
    "v10" DOUBLE PRECISION,
    "pressure" DOUBLE PRECISION,
    "latitude" DOUBLE PRECISION,
    "longitude" DOUBLE PRECISION
  )
);
```

8. Now we copy the sample data from the S3 bucket. Paste the copy from statement into the "Console" and click "Execute"

```
COPY demo.climate_data FROM 's3://crate-tutorials/realtime-demo/climate_data.json';
```

View Grafana dashboards



Your dashboard should be accessible at:

<http://ec2-13-218-101-16.compute-1.amazonaws.com:8888/d/adgxp2x/cratedb-real-time-demo?orgId=1&from=2025-08-09T00:00:00.000Z&to=2025-08-16T23:59:59.000Z&timezone=browser>

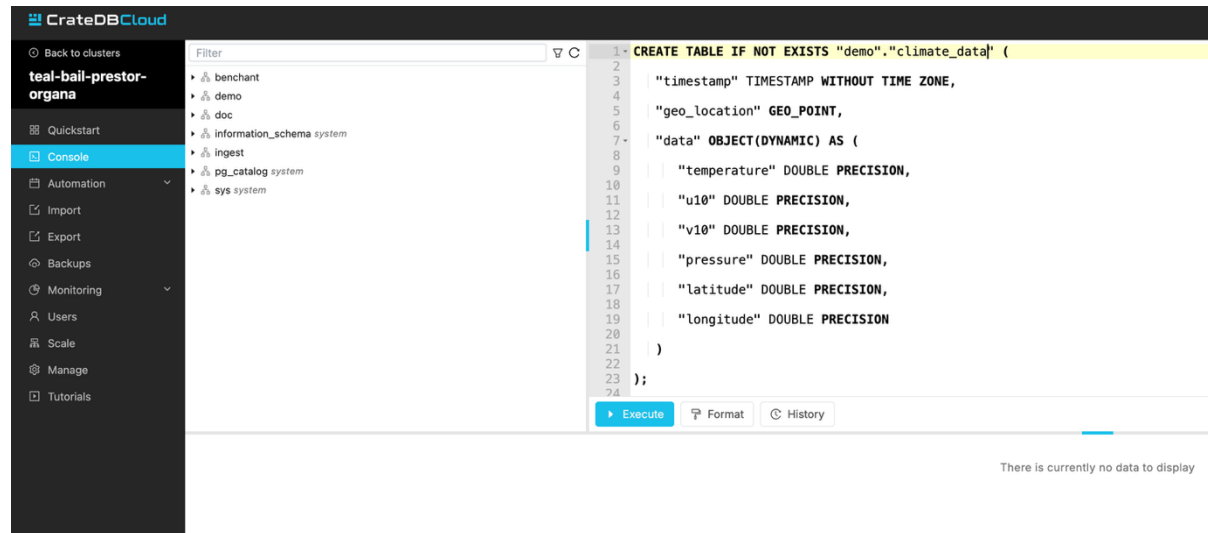
To login, use the following Grafana username and passwords

cratedb_demo/your_secure_password

All complete!

Exercises – Query

To run the exercises, you need to go to “Console” paste the queries in and click on “Execute”



Try some queries to see the flexibility in the data model

What is the time range?

```
SELECT
  min("timestamp") AS start_ts,
  max("timestamp") AS end_ts
FROM
  demo.climate_data;
```

Timestamps in CrateDB are always stored without a timezone. If the field is defined WITH a timezone, then on ingest the timestamp is adjusted accordingly to UTC.

In this data, it's defined without a timezone. It's defined as the number of milliseconds since the Unix epoch (1970-01-01T00:00:00Z). For a detailed guide to all the available datatypes see:

<https://cratedb.com/docs/crate/reference/en/latest/general/ddl/data-types.html#timestamp>

Note: The CrateDB console will display both the number of milliseconds and the human readable date and time.

Distribution of temperature

```
SELECT
  min(data['temperature']) AS min_t,
  avg(data['temperature']) AS avg_t,
  max(data['temperature']) AS max_t
```

```
FROM
  demo.climate_data;
```

This is a straightforward aggregation query. It's running a query across all data in the table and returning the minimum, maximum and average temperatures.

Notice the **AS**, which is an alias. Normally fields in an aggregate will be unnamed expressions, but with recent versions of CrateDB it will generate an alias automatically. However, it's still recommended to give a name.

Average temperature per day

```
SELECT
  date_trunc('day', "timestamp") AS day,
  min(data['temperature']) AS min_temp,
  max(data['temperature']) AS max_temp
FROM
  demo.climate_data
GROUP BY
  1
ORDER BY
  1;
```

This query introduces a new function - **date_trunc**. This returns a timestamp value *truncated* to a specific precision, in this case to precision of a day and effectively ignores the HH:MM:SS of any timestamp. This gives us a per-day aggregate figure. To get information on more functions like this, see:

<https://cratedb.com/docs/crate/reference/en/latest/general/builtins/scalar-functions.html>

10 hottest points

```
SELECT
  "timestamp",
  geo_location,
  data['temperature'] AS temp
FROM
  demo.climate_data
ORDER BY
  temp DESC
LIMIT
  10;
```

To give us the top 10 hottest points, we use **ORDER BY** (to order by temperature in descending order) and **LIMIT** to only return 10 rows.

The **SELECT** part of the query returns us the timestamp (i.e. when), the location (as a **GEO_POINT** co-ordinate, which is explained later) and the temperature from the **data** JSON object.

Are we missing any data points?

```
SELECT
  count(*) FILTER (WHERE data['temperature'] IS NULL) AS missing_temp,
  count(*) FILTER (WHERE data['pressure'] IS NULL) AS missing_pressure,
  count(*) FILTER (WHERE geo_location IS NULL) AS missing_geo
FROM
  demo.climate_data;
```

As you can see, nothing is missing in the included data. How does this query work?

FILTER is a window function. It performs a calculation across a set of related rows defined by an expression (or multiple expressions). In this case we are **COUNT**ing the number of rows where one of **data['temperature']**/**data['pressure']** or **geo_location** is NULL (i.e. undefined).

Window functions are very powerful, and it's very much recommended to read the following webpage to learn more about their capabilities.

<https://cratedb.com/docs/crate/reference/en/latest/general/builtins/window-functions.html>

Let's add a point with a missing temperature and re-run:

```
INSERT INTO demo.climate_data (timestamp, geo_location, data) VALUES
(1754956800000, [8.788311111111111, 54.903], {"longitude" = 8.788311111111111,
"latitude" = 54.903, "u10" = 4.472952365875244, "v10" = -1.3958832025527954,
"pressure" = 102426.1015625});
```

If you now re-run the missing data point query, it will show:

```
missing_temp missing_pressure missing_geo
1           0                0
```

Now let's update the record with a valid temperature:

```
UPDATE demo.climate_data
SET
  data['temperature'] = 16.8
WHERE
  timestamp = 1754956800000
  AND geo_location = [8.788311111111111, 54.903];
```

Re-run the missing data point query, and you'll see it now shows zero again.

Wind speed calculations

This query will calculate min/max and avg wind speeds (in metres per second) using Pythagoras' theorem to calculate the diagonal (hypotenuse) value:

$$\sqrt{(x^2 + y^2)}$$

```
SELECT
  min(sqrt(power(data['u10'], 2) + power(data['v10'], 2))) AS min_ws,
  avg(sqrt(power(data['u10'], 2) + power(data['v10'], 2))) AS avg_ws,
  max(sqrt(power(data['u10'], 2) + power(data['v10'], 2))) AS max_ws
FROM demo.climate_data;
```

The `power` function returns the field to the specified power, 2 in this case, i.e. it squares the field value. The `sqrt` functions returns the square root. These are part of the CrateDB scalar functions, and more information is available here:

<https://cratedb.com/docs/crate/reference/en/latest/general/builtins/scalar-functions.html>

Can we optimise this query? The mathematical expression is being calculated 3 times and this is inefficient. We can use a CTE (common table expression) to optimise this:

```
WITH wind_speed AS (
  SELECT
    sqrt(power(data['u10'], 2) + power(data['v10'], 2)) AS ws
  FROM demo.climate_data
)
SELECT
  MIN(ws) AS min_ws,
  AVG(ws) AS avg_ws,
  MAX(ws) AS max_ws
FROM wind_speed;
```

Now we are only calculating once. For more information on common table expressions, see:

<https://cratedb.com/querying/common-table-expressions-ctes>

What are the top 10 windiest observations?

```
SELECT
  "timestamp",
  geo_location,
  sqrt(power(data['u10'], 2) + power(data['v10'], 2)) AS wind_speed
FROM
  demo.climate_data
ORDER BY
  wind_speed DESC
LIMIT 10;
```

This uses the wind speed calculation as detailed in the previous query.

Wind direction calculated using trigonometry

This query will show the wind direction in degrees.

```
SELECT
  date_trunc('hour', "timestamp") AS hour,
  avg(sqrt(power(data['u10'], 2) + power(data['v10'], 2))) AS avg_speed,
  avg(
    mod(degrees(atan2(data['u10'], data['v10'])) + 360, 360)
  ) AS avg_dir_deg
FROM demo.climate_data
GROUP BY 1
ORDER BY 1;
```

The function `mod` has been introduced here, this refers to the **modulus** operation (`mod` is a shortened alias to **modulus**). This function returns the remainder of a divide operation; in this case we use it so that the angle in degrees is always between 0 and 360, i.e. an angle of 370 would return 10.

The other new function is `atan2`, which is a trigonometry function. All these are covered in the scalar functions guide:

<https://cratedb.com/docs/crate/reference/en/latest/general/builtins/scalar-functions.html#scalar-modulus>

Exercise: try returning the result in radians (ensure you update the modulo operation, the `pi` function will be useful here!)

Exercises - Geo

How to extract latitude & longitude from a GEO_POINT

You can use the `latitude` and `longitude` functions to return the co-ordinates in a `GEO_POINT` datatype. This example will return the bounding box for all loaded data:

```
SELECT
  min(longitude(geo_location)) AS min_longitude,
  max(longitude(geo_location)) AS max_longitude,
  min(latitude(geo_location)) AS min_latitude,
  max(latitude(geo_location)) AS max_latitude
FROM
  demo.climate_data;
```

Let's use the result of this to work out how many data points there are in a subset of the data, take this query and fill in some correct values based on the result of the above query.

```
SELECT count(*) AS points
FROM demo.climate_data
WHERE latitude(geo_location) BETWEEN 0.0 AND 0.0
      AND longitude(geo_location) BETWEEN 0.0 AND 0.0;
```

If you get it right, this should return a non-zero count. If it's zero, check your query and try again.

More information on CrateDB's geospatial capabilities is detailed here:

<https://cratedb.com/docs/guide/start/modelling/geospatial.html>

A more powerful way of doing this is to use the `within` function. This allows us to find points that lie within a polygon, which gives us the flexibility to use shapes other than a rectangle:

```
SELECT count(*) AS points
FROM demo.climate_data
WHERE within(
  geo_location,
  {
    "type": "Polygon",
    "coordinates": [[
      [0.0, 0.0],
      [0.0, 1.0],
      [1.0, 1.0],
      [1.0, 0.0],
      [0.0, 0.0]
    ]]
  }
);
```

The list of points is an array, in the form:

```
[longitude1, latitude1],  
[longitude2, latitude2] ,  
[longitude3, latitude3] ,  
[longitude4, latitude4] ,  
[longitude1, latitude1]
```

Which effectively *defines* a rectangular bounding box.

Note: Don't be afraid to ask for some guidance from one of the CrateDB team.

Distance calculations

CrateDB also supports distance calculations, the following query gives the nearest points to Berlin, Germany (approximate location is 52.5N, 13.4E)

```
SELECT  
  geo_location,  
  distance(geo_location, 'POINT(13.4 52.5)') AS metres,  
  data['temperature'] AS temp  
FROM demo.climate_data  
GROUP BY geo_location, metres, temp  
ORDER BY metres ASC  
LIMIT 50;
```

Exercise: Try changing the location from Berlin to some other location.

Exercises – Correlation

Is pressure and temperate related?

Is there a relationship between temperature and atmospheric pressure (at ground level)?

Run this query and decide for yourself:

```
SELECT
  geo_location,
  avg(data['temperature']) AS avg_temp,
  avg(data['pressure']) AS avg_pressure
FROM demo.climate_data
GROUP BY 1
ORDER BY 2 DESC;
```

Exercise: Is there a way to improve this query? Maybe a combination of temperature variance from average and air pressure from 100,000 (1 atmosphere)?

Is pressure and wind speed related?

Is there a relationship between temperature and wind speed?

```
SELECT
  geo_location,
  avg(data['temperature']) AS avg_temp,
  avg(data['pressure']) AS avg_pressure,
  avg(sqrt(power(data['u10'], 2) + power(data['v10'], 2))) AS wind_speed
FROM demo.climate_data
GROUP BY 1
ORDER BY 3 DESC;
```

Exercise: Try altering the ordering, either **ASC**ending or **DESC**ending, or using a different column (or combination).

Exercise – JSON object model

Insert a new value into the dataset.

```
INSERT INTO demo.climate_data (timestamp, geo_location, data) VALUES
(123, [8.788311111111111, 54.903], {"longitude" = 8.788311111111111,
"latitude" = 54.903, "temperature" = 16.868310546875023, "u10" =
4.472952365875244, "v10" = -1.3958832025527954, "pressure" =
102426.1015625});
```

This is using the existing schema. What if we want to add a new humidity value to our dataset, do we need to recreate the table?

In CrateDB we can define object columns (i.e. JSON) as one of three different types:

OBJECT(STRICT) – The schema for the object is fixed

OBJECT(DYNAMIC) – The schema is dynamic, and new attributes can be added

OBJECT(IGNORED) – The JSON is stored ignoring the datatypes of the attributes and not indexing the field

<https://cratedb.com/blog/handling-dynamic-objects-in-cratedb>

When you created this table you set the **data** column to be DYNAMIC, so we can add additional attributes easily, and these attributes will be indexed.

Add a new key in the JSON object without the need to modify the schema

```
INSERT INTO demo.climate_data (timestamp, geo_location, data) VALUES
(123, [8.788311111111111, 54.903], {"humidity" = 43.4, "longitude" =
8.788311111111111, "latitude" = 54.903, "temperature" =
16.868310546875023, "u10" = 4.472952365875244, "v10" = -
1.3958832025527954, "pressure" = 102426.1015625});
```

Now try to query the new key:

```
SELECT
  data['humidity']
FROM
  demo.climate_data
WHERE
  data['humidity'] IS NOT NULL;
```

This makes it very easy to extend the data model, ideal for IoT applications where additional sensor information may become available.

And of course, with CrateDB all of this additional data will be indexed for great query performance.

Exercise: Try adding additional columns of data or more data and add this view to the Grafana dashboards.